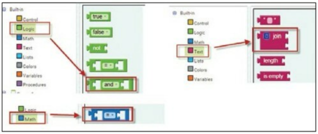

Figure 16: Other built-in blocks

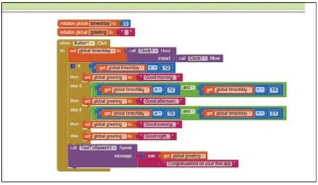

Figure 17: The complete code

as shown in Figure 15.

> **Note:** It is natural for a beginner to try to drag the 'else-if' into the main 'if then' block below the configuration area. That is a futile exercise, which will not change anything and not give any error message either.

To complete our work, we need to use the *Control->and* block, the *Math->compare* block and a *Text->join* block, so that we can greet and then give the congratulatory message. The positions of these blocks are shown in Figure 16.

Also, to get a pause between the greeting and the congratulatory message we can add a few 'full stops' to the message. The complete blocks' code looks like what's shown in Figure 17.

A few more notes are provided below:

- There are warnings and error counts shown all the time in the blocks view. Make sure you keep checking this to understand if something is going wrong.
- If you click on any item from the built-in palette by mistake, removing the same can be done by simply selecting the block and 'deleting' it using the 'Delete' key. You can also drag it into the dustbin shown in the GUI and drop it when its cover opens.
- Calling *TextToSpeech1.Speak* multiple times back-to-back produced the speech only for the last invocation during my attempts. Hence, I went for a simpler join to get multiple messages in.
- Testing our app for different times of the day is not easy without some innovation. You could think of changing the clock in the computer and then testing it using the emulator.
- There is a bit of learning to be done to get logical code done in the blocks editor, especially if you have become used to writing code using a text editor. Once you cut your teeth with a few applications, you should be fine to try out more. Being patient initially could help you with this.
- The project resides in the cloud all the time. When you are not connected to the Internet, you may be able to make changes to your design blocks to test your changes. But you will not be able to change to 'Designer view' without being connected and, once you do so, you may be surprised to find your older blocks' changes have not been saved. So it is recommended to make a local copy of the app; it will save as an *.aia* file on your local drive.
- Also, it seems that the AI 2 interface still supports the earlier Android look and feel. There may be tools which could upgrade your Android app to the latest look and feel. Try them out.
- There are also ways by which you can publish your app in the AI 2 store and also in the Android Play Store. END

> **Note:** If you do succeed in building an app after reading this article, the author would love to hear from you.

References

[1] IDC smartphone report Q1 2015 – *http://www.idc.com/prodserv/smartphone-os-market-share.jsp*
[2] Android devices statistics – *http://www.statista.com/topics/876/android/*
[3] Developer payout in 2014 to Android app developers *http://www.theguardian.com/technology/2015/jan/28/android-ios-app-downloads-revenues-app-annie-google-play-app-store*
[4] MIT App Inventor 2 - *http://ai2.aappinventor.mit.edu*
[5] Wiki entry for Android App Inventor – *https://en.wikipedia.org/wiki/App_Inventor_for_Android*
[6] What's new in MIT AI 2 - *http://appinventor.mit.edu/explore/ai2/whats-new.html*
[7] Instructions to set up Emulator - *http://appinventor.mit.edu/explore/ai2/setup-emulator.html*
[8] Connecting with your phone - *http://appinventor.mit.edu/explore/ai2/setup-device-wifi.html*
[9] Publishing in AI 2 - *http://www.appinventor.org/content/CourseInABox/Intro/PublishingHowTo*
[10] Publish to Android Play Store - *http://www.crucialthought.com/2011/05/15/publishing-an-app-inventor-app-to-the-android-market/*

By: T.V. Krishnamurthy

The author has more than 15 years of experience in the telecom industry and has authored the book "4g: deployment option and operational implications". He is also a python evangelist. He lives and works out of Bengaluru in India.